1. Overview This document outlines the integration protocols between the User Presentation Layer (Frontend) and the Server-Side Application Layer (Backend). The backend modules have been fully normalized and equipped with core business logic, including database transactions (ACID compliance), an overlapping schedule prevention algorithm (with a 30-minute inspection buffer), and an automated financial penalty settlement system. Frontend developers must adhere to the following payload structures and HTTP methods to ensure seamless data handshakes with the backend database.

---

2. API Interface: Venue Booking Submission

- **Module:** Reservation Engine
- **Target Endpoint:** `/actions/process_booking.php`
- **HTTP Method:** `POST`
- **Description:** This endpoint receives the user's booking request, validates time-slot availability against existing records, and simultaneously generates a reservation record (`bookings` table) and a financial deposit record (`payments` table) using secure Database Transactions.

Payload Requirements (HTML Form `<input>` name attributes): The frontend form must submit the following parameters exactly as named:

Parameter Name	Data Type	Description & Frontend Implementation Guide
`user_id`	Integer	The ID of the currently logged-in student. *(Note: Use a hidden input field for testing; this will later be migrated to secure `$_SESSION` retrieval).*
`venue_id`	Integer	The unique identifier of the selected venue. Passed via `<select>` dropdown or hidden input.
`booking_date`	Date	The intended date of usage. Format: `YYYY-MM-DD` (e.g., `<input type="date">`).
`start_time`	Time	The start time of the reservation. Format: `HH:MM` (e.g., `<input type="time">`).
`end_time`	Time	The end time of the reservation. Format: `HH:MM` (e.g., `<input type="time">`).

Parameter Name	Data Type	Description & Frontend Implementation Guide
`deposit_paid`	Decimal	The base deposit amount associated with the selected venue. Must be retrieved dynamically from the `venues` table and passed via hidden input.

Expected Backend Response:

- **Success:** The system will commit the transaction and return a success message with the generated Booking ID.
- **Failure (Conflict):** If a time conflict or buffer overlap is detected, the transaction is rolled back, and an error prompt is returned to the user.

---

3. API Interface: Venue Catalog Retrieval (Status Filtering) *(NEWLY ADDED)*

- **Module:** Frontend Venue Display Catalog
- **Action Type:** Data Retrieval (SQL `SELECT`)
- **Description:** The backend has introduced a dynamic status management system for venues, including automated lockdowns for major damages and a "Soft Delete" mechanism for decommissioned resources. To prevent users from booking unavailable facilities, the frontend must implement strict data filtering.

Venue Status Data Dictionary (`venues.status`):

- `Available`: The venue is operational and open for booking. **(Must be displayed to the user)**.
- `Maintenance`: The venue is temporarily locked by the system due to major damage reported during post-usage inspection. **(Must be hidden)**.
- `Closed`: The venue has been permanently decommissioned or soft-deleted by the administrator to preserve financial audit trails. **(Must be hidden)**.

Required SQL Implementation for Frontend:

When rendering the venue selection dropdown or catalog, the frontend developer must apply the following SQL constraint:

```SQL
SELECT * FROM venues WHERE status = 'Available';
```

---

4. System Interface: Manual Venue Return (State Transition)

- **Module:** Post-Usage Lifecycle Management
- **Action Type:** Frontend Database Update
- **Description:** To optimize venue availability and streamline the administrator's inspection workflow, users are provided with a "Manual Return" feature. This allows users to voluntarily release a venue before their scheduled end time.

Frontend Implementation Guide:

- **UI Placement:** Render a "Return Venue" button on the user's "My Bookings" history page.
- **Trigger Condition:** The button should only be visible for bookings where `booking_status = 'Approved'`.
- **Execution Logic:** Upon clicking the button, the frontend application must execute an `UPDATE` query to transition the booking state.

Required SQL Execution:

```SQL
UPDATE bookings
SET booking_status = 'Returned'
WHERE booking_id = [Target_Booking_ID] AND user_id = [Current_User_ID];
```

Post-Action Workflow: Once the state is successfully transitioned to `'Returned'`, the backend's dynamic Time-Based Filtering algorithm will automatically route this specific record into the Administrator's Pending Inspection Queue.

---

5. Frontend Development Constraints & Best Practices

1. **Client-Side Validation:** The frontend must implement basic HTML5 or JavaScript validation to ensure that the `start_time` is strictly earlier than the `end_time` before submitting the `POST` request.
2. **Native Implementation:** In accordance with project requirements, the integration must rely on native HTML forms and PHP `$_POST` handling. The use of third-party form builders, Adobe Dreamweaver data behaviors, or unauthorized code generators is strictly prohibited.